ASSIGNMENT - 02 - CO2

(Control Flow, Strings, Functions and Recursion)

COURSE NAME : Python Programming

COURSE CODE : CSA0825

COURSE FACULTY : Hariramachandran T

SUBMITTED BY : G.anushka(1912260668)

GITUB LINK:https://github.com/anushkapearl21-jpg/CSA0825-co2.git

Q1 – Decisions with Many Boundaries

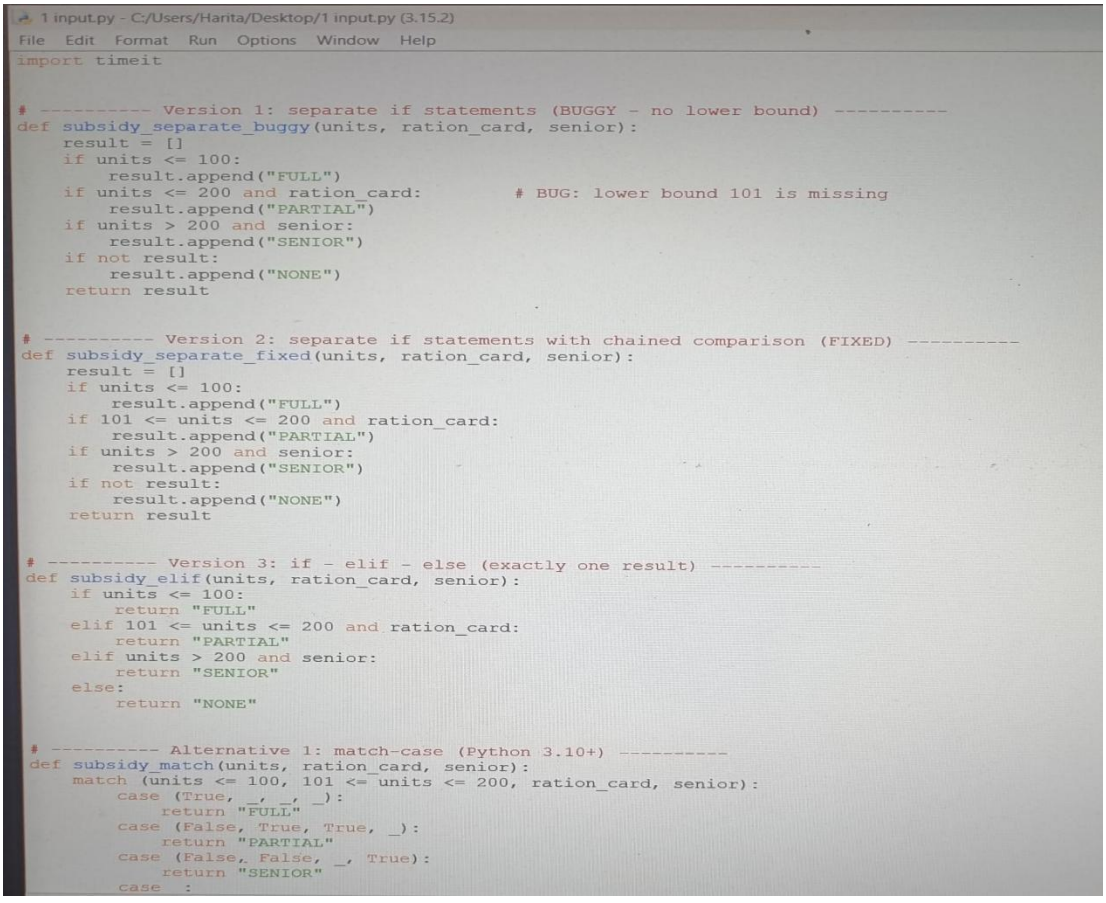
Aim:

To implement the State Electricity Subsidy rule using separate if statements and using if–elif–else, test every boundary (100, 101, 200, 201 units), and show why if–elif–else always gives exactly one decision.

Algorithm:

1. Read units consumed, whether the household has a ration card, and whether it has a member above 80 years.
2. If units <= 100, the decision is FULL subsidy.
3. Else if 101 <= units <= 200 and the household has a ration card, the decision is PARTIAL subsidy.
4. Else if units > 200 and the household has a member above 80, the decision is SENIOR subsidy.
5. Otherwise the decision is NO subsidy.
6. Print the decision. Repeat with separate if statements & compare the two results.

Input:



```
1 input.py - C:/Users/Harita/Desktop/1 input.py (3.15.2)
File  Edit  Format  Run  Options  Window  Help
import timeit


# ---------- Version 1: separate if statements (BUGGY - no lower bound) ----------
def subsidy_separate_buggy(units, ration_card, senior):
    result = []
    if units <= 100:
        result.append("FULL")
    if units <= 200 and ration_card:        # BUG: lower bound 101 is missing
        result.append("PARTIAL")
    if units > 200 and senior:
        result.append("SENIOR")
    if not result:
        result.append("NONE")
    return result


# ---------- Version 2: separate if statements with chained comparison (FIXED) ----------
def subsidy_separate_fixed(units, ration_card, senior):
    result = []
    if units <= 100:
        result.append("FULL")
    if 101 <= units <= 200 and ration_card:
        result.append("PARTIAL")
    if units > 200 and senior:
        result.append("SENIOR")
    if not result:
        result.append("NONE")
    return result


# ---------- Version 3: if - elif - else (exactly one result) ----------
def subsidy_elif(units, ration_card, senior):
    if units <= 100:
        return "FULL"
    elif 101 <= units <= 200 and ration_card:
        return "PARTIAL"
    elif units > 200 and senior:
        return "SENIOR"
    else:
        return "NONE"


# ---------- Alternative 1: match-case (Python 3.10+) ----------
def subsidy_match(units, ration_card, senior):
    match (units <= 100, 101 <= units <= 200, ration_card, senior):
        case (True, _, _, _):
            return "FULL"
        case (False, True, True, _):
            return "PARTIAL"
        case (False, False, _, True):
            return "SENIOR"
        case _:
```

```
# ---------- Alternative 2: table-driven (slabs) ----------
SLABS = [
    (100, lambda ration, senior: "FULL"),
    (200, lambda ration, senior: "PARTIAL" if ration else "NONE"),
    (float("inf"), lambda ration, senior: "SENIOR" if senior else "NONE"),
]


def subsidy_table(units, ration_card, senior):
    for limit, rule in SLABS:
        if units <= limit:
            return rule(ration_card, senior)


# ---------- Test: eight households including every boundary ----------
households = [  # (units, ration card, member above 80)
    (50, True, False), (100, False, False), (100, True, False), (101, True, False),
    (101, False, False), (200, True, False), (201, False, False), (201, False, True),
]
predicted = ["FULL", "FULL", "FULL", "PARTIAL", "NONE", "PARTIAL", "NONE", "SENIOR"]

print(f"{'units':>5} {'ration':>6} {'age>80':>6} | {'predicted':9} | {'buggy separate-if':22} | "
      f"{'fixed separate-if':18} | {'elif':7} | {'match':7} | {'table':7}")
for h, p in zip(households, predicted):
    print(f"{h[0]:>5} {str(h[1]):>6} {str(h[2]):>6} | {p:9} | {str(subsidy_separate_buggy(*h)):22} | "
          f"{str(subsidy_separate_fixed(*h)):18} | {subsidy_elif(*h):7} | {subsidy_match(*h):7} | {subsidy_table(*h):7}")

all_ok = all(subsidy_elif(*h) == p == subsidy_match(*h) == subsidy_table(*h)
             for h, p in zip(households, predicted))
print("elif / match / table agree with prediction:", all_ok)

# ---------- Time per decision ----------
print("\nTime for 200,000 decisions (units=250, last slab, worst case):")
for f in (subsidy_elif, subsidy_match, subsidy_table):
    t = timeit.timeit(lambda: f(250, False, True), number=200_000)
    print(f"{f.__name__:<15} {t:.4f} s")
```

Output:

```
IDLE Shell 3.15.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.15.2 (tags/v3.15.2:5e58eaa, Oct 25 2026, 17:27:03) [MSC v.1951 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
================== RESTART: C:/Users/Harita/Desktop/1 input.py ==================
units ration age>80 | predicted | buggy separate-if      | fixed separate-if  | elif    | match   | table
   50   True  False | FULL      | ['FULL', 'PARTIAL']    | ['FULL']           | FULL    | FULL    | FULL
  100  False  False | FULL      | ['FULL']               | ['FULL']           | FULL    | FULL    | FULL
  100   True  False | FULL      | ['FULL', 'PARTIAL']    | ['FULL']           | FULL    | FULL    | FULL
  101   True  False | PARTIAL   | ['PARTIAL']            | ['PARTIAL']        | PARTIAL | PARTIAL | PARTIAL
  101  False  False | NONE      | ['NONE']               | ['NONE']           | NONE    | NONE    | NONE
  200   True  False | PARTIAL   | ['PARTIAL']            | ['PARTIAL']        | PARTIAL | PARTIAL | PARTIAL
  201  False  False | NONE      | ['NONE']               | ['NONE']           | NONE    | NONE    | NONE
  201  False   True | SENIOR    | ['SENIOR']             | ['SENIOR']         | SENIOR  | SENIOR  | SENIOR
elif / match / table agree with prediction: True

Time for 200,000 decisions (units=250, last slab, worst case):
subsidy_elif    0.0175 s
subsidy_match   0.0381 s
subsidy_table   0.0259 s
>>>
```

Predictions before running (eight households):

Units	Ration card	Member above 80	Predicted decision
50	Yes	No	FULL
100	No	No	FULL
100	Yes	No	FULL
101	Yes	No	PARTIAL
101	No	No	NONE

Units	Ration card	Member above 80	Predicted decision
200	Yes	No	PARTIAL
201	No	No	NONE
201	No	Yes	SENIOR

Why separate if statements can give two results?

Every separate if is tested independently, so more than one condition can be true for the same household. In the buggy version the second condition was **units <= 200 and ration card** (no lower bound), so a household with 50 units and a ration card satisfied both the first and the second if and received FULL and PARTIAL, i.e. two subsidies (see the buggy column in the output). In if–elif–else the conditions are tested in order and as soon as one branch runs, all the remaining branches are skipped, so exactly one result is produced.

Design decision:

I choose a flat if–elif chain with **combined Boolean conditions (and)**. Reasons: (1) each branch reads like one sentence of the policy, so it is easy to check against the rule; (2) the "no subsidy" case appears in two places (101–200 units without ration card, and above 200 units without a senior member) and is handled once by the final else; with nested if the else would have to be repeated in each nest; (3) if a slab or condition changes, only one line is edited and the nesting depth does not grow.

Alternative concept and complexity:

An alternative approach is to use nested if statements instead of an if–elif–else chain. Each condition is checked inside another condition. However, nested if statements can make the program longer and harder to understand when more conditions are added. Both approaches take O(1) time because only a fixed number of conditions are checked, and O(1) extra space because no additional data structures are required. The if–elif–else approach is preferred because it is simpler, more readable and avoids multiple decisions being executed.

Q2 – Loops that Count Attempts

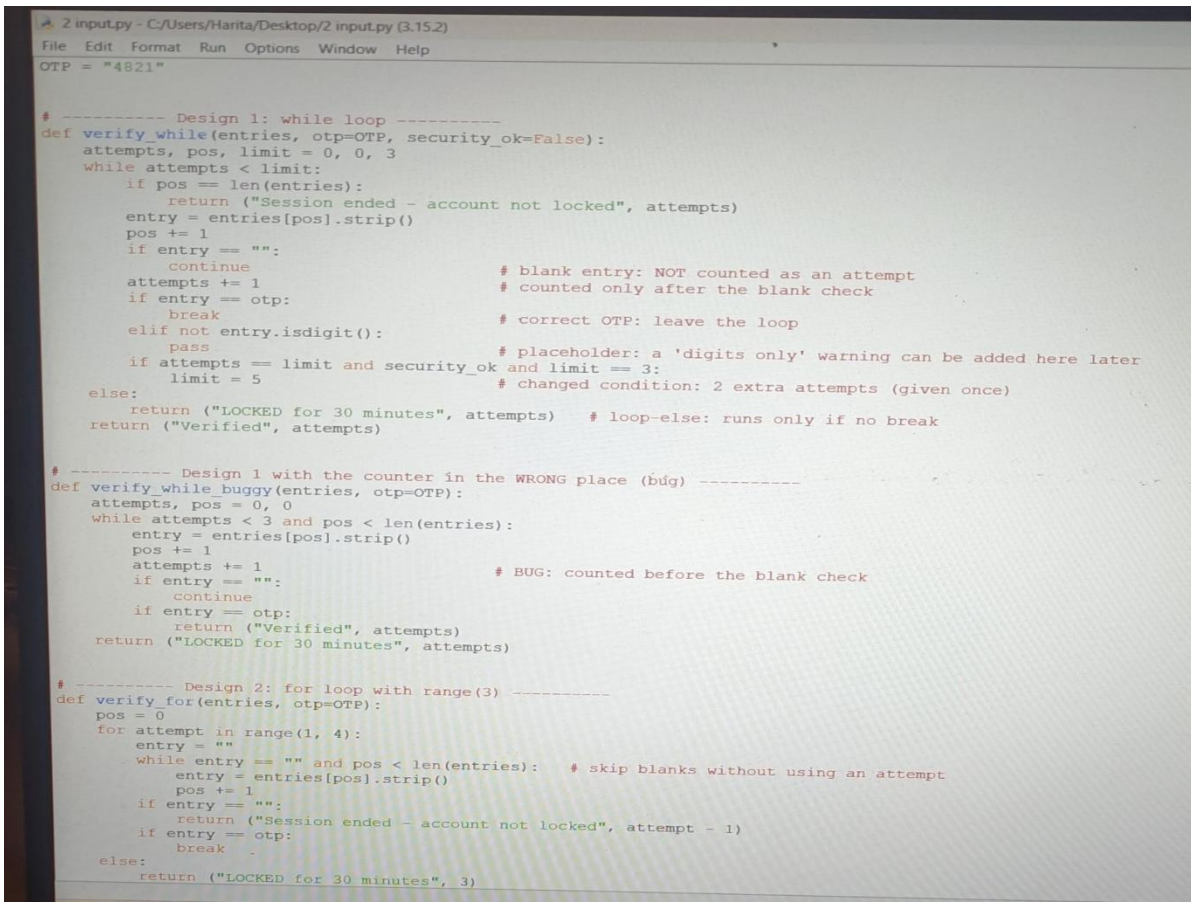
Aim:

To design the three-attempt OTP verification of a bank kiosk using a while loop and a for loop with range(3), using break, continue, pass and the loop else clause to lock the account, and to handle the changed condition (two extra attempts).

Algorithm:

1. Set attempts = 0 and limit = 3.
2. While attempts < limit, read the next entry.
3. If the entry is blank, use continue (a blank entry is not counted).
4. Otherwise increase attempts by 1. If the entry equals the OTP, use break (verified).
5. If the entry is not a number, pass (placeholder where a warning can be added later).
6. If the loop finishes without break, the loop else clause locks the account for 30 minutes.
7. Changed condition: when the third attempt fails and the security question was answered correctly, set limit = 5 once.

Input:



```
    return ("Verified", attempt)


# ---------- Alternative: recursion ----------
def verify_rec(entries, otp=OTP, pos=0, attempts=0, limit=3):
    if attempts == limit:
        return ("LOCKED for 30 minutes", attempts)
    if pos == len(entries):
        return ("Session ended - account not locked", attempts)
    entry = entries[pos].strip()
    if entry == "":
        return verify_rec(entries, otp, pos + 1, attempts, limit)
    if entry == otp:
        return ("Verified", attempts + 1)
    return verify_rec(entries, otp, pos + 1, attempts + 1, limit)


tests = {
    "correct on 3rd attempt":          ["1111", "", "2222", "4821"],
    "three wrong -> lock":             ["1", "2", "3"],
    "blanks first, correct 3rd":       ["", "", "1", "2", "4821"],
    "blank then 2 wrong (bug demo)":   ["", "1111", "2222", "4821"],
}

for name, e in tests.items():
    print(f"{name:32} while: {verify_while(e)}  | for: {verify_for(e)} | recursion: {verify_rec(e)}")

print("\nBuggy counter on 'blank then 2 wrong':", verify_while_buggy(tests["blank then 2 wrong (bug demo)"]))

print("\nChanged condition (security question answered correctly -> 2 extra attempts):")
e = ["1", "2", "3", "9", "4821"]
print("  4th wrong, 5th correct, security_ok=True :", verify_while(e, security_ok=True))
print("  same entries, security_ok=False          :", verify_while(e, security_ok=False))
print("  for-range(3) design cannot do this without changing range(3) itself")
```

Output:

```
IDLE Shell 3.15.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.15.2 (tags/v3.15.2:5e58eaa, Oct 25 2026, 17:27:03) [MSC v.1951 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/Harita/Desktop/2 input.py ==========
correct on 3rd attempt          while: ('Verified', 3)  | for: ('Verified', 3) | recursion: ('Verified', 3)
three wrong -> lock             while: ('LOCKED for 30 minutes', 3)  | for: ('LOCKED for 30 minutes', 3) | recursion: ('LOCKED for 30 minutes', 3)
blanks first, correct 3rd       while: ('Verified', 3)  | for: ('Verified', 3) | recursion: ('Verified', 3)
blank then 2 wrong (bug demo)   while: ('Verified', 3)  | for: ('Verified', 3) | recursion: ('Verified', 3)

Buggy counter on 'blank then 2 wrong': ('LOCKED for 30 minutes', 3)

Changed condition (security question answered correctly -> 2 extra attempts):
  4th wrong, 5th correct, security_ok=True : ('Verified', 5)
  same entries, security_ok=False          : ('LOCKED for 30 minutes', 3)
  for-range(3) design cannot do this without changing range(3) itself
>>>
```

When is while more natural than for?

The number of entries read is not fixed: blank entries do not count and the limit can change to five. A while loop tests the condition attempts < limit every time, so the limit can be changed while the loop is running. A for loop over range(3) fixes the number of repetitions in advance and cannot use continue for a blank entry without losing an attempt (the for version needs an inner while loop to skip blanks).

What bug appears if the counter is updated in the wrong place?

if attempts is increased before the blank check, a blank entry is counted as an attempt. In the output, the entries blank, 1111, 2222, 4821 are verified correctly by the right design (3 real

attempts) but the buggy design locks the account after only two real attempts. If the counter is not updated at all, the loop never ends.

Roles:

break leaves the loop on the correct OTP and skips the else clause; continue skips a blank entry without counting; pass does nothing and keeps the empty branch legal; the loop else runs only when the loop ended without break, so it is exactly the “three failures” case.

Changed condition:

The while design needs one change: a variable limit that becomes 5 after a correct security answer. The for range(3) design must be rewritten, because the decision is taken after the third failure, when range(3) has already been fixed.

Alternative Concept and Complexity Analysis:

verify_rec(entries, otp, pos, attempts, limit) calls itself for the next attempt (see the program). Time is O(a) for a attempts in all three versions. Stack space is O(a) for recursion (one frame per attempt) and O(1) for the loops. With a at most 5 this is harmless, but the loop is simpler and safer.

Q3 – Vehicle Registration Number Validator

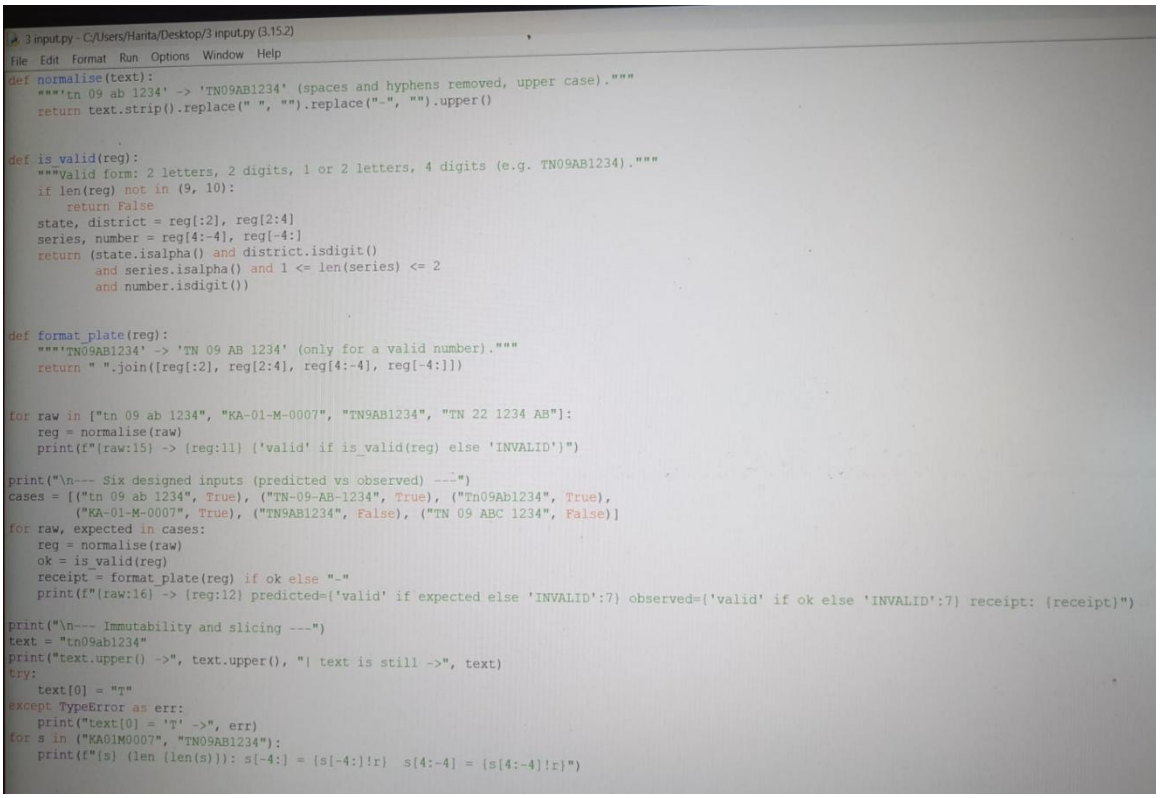
Aim:

To normalize, validate and format vehicle registration numbers using only string operations (indexing, slicing, len, upper, replace, strip, isalpha, isdigit, join) and no regular expressions.

Algorithm:

1. normalize: remove leading/trailing spaces, remove all spaces and hyphens, convert to upper case.
2. is_valid: if the length is not 9 or 10, return False.
3. Slice the number: state = reg[:2], district = reg[2:4], series = reg[4:-4], number = reg[-4:].
4. Return True only if state.isalpha(), district.isdigit(), series.isalpha() with length 1 or 2, and number.isdigit().
5. format_plate: join the four slices with a single space, e.g. TN 09 AB 1234.
6. Test six designed inputs (two should be rejected) and the four inputs given in the question.

Input:



```
3 input.py - C:/Users/Harita/Desktop/3 input.py (3.15.2)
File  Edit  Format  Run  Options  Window  Help
def normalise(text):
    """'tn 09 ab 1234' -> 'TN09AB1234' (spaces and hyphens removed, upper case)."""
    return text.strip().replace(" ", "").replace("-", "").upper()


def is_valid(reg):
    """Valid form: 2 letters, 2 digits, 1 or 2 letters, 4 digits (e.g. TN09AB1234)."""
    if len(reg) not in (9, 10):
        return False
    state, district = reg[:2], reg[2:4]
    series, number = reg[4:-4], reg[-4:]
    return (state.isalpha() and district.isdigit()
            and series.isalpha() and 1 <= len(series) <= 2
            and number.isdigit())


def format_plate(reg):
    """'TN09AB1234' -> 'TN 09 AB 1234' (only for a valid number)."""
    return " ".join([reg[:2], reg[2:4], reg[4:-4], reg[-4:]])


for raw in ["tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234", "TN 22 1234 AB"]:
    reg = normalise(raw)
    print(f"{raw:15} -> {reg:11} {'valid' if is_valid(reg) else 'INVALID'}")

print("\n--- Six designed inputs (predicted vs observed) ---")
cases = [("tn 09 ab 1234", True), ("TN-09-AB-1234", True), ("Tn09Ab1234", True),
         ("KA-01-M-0007", True), ("TN9AB1234", False), ("TN 09 ABC 1234", False)]
for raw, expected in cases:
    reg = normalise(raw)
    ok = is_valid(reg)
    receipt = format_plate(reg) if ok else "-"
    print(f"{raw:16} -> {reg:12} predicted={'valid' if expected else 'INVALID':7} observed={'valid' if ok else 'INVALID':7} receipt: {receipt}")

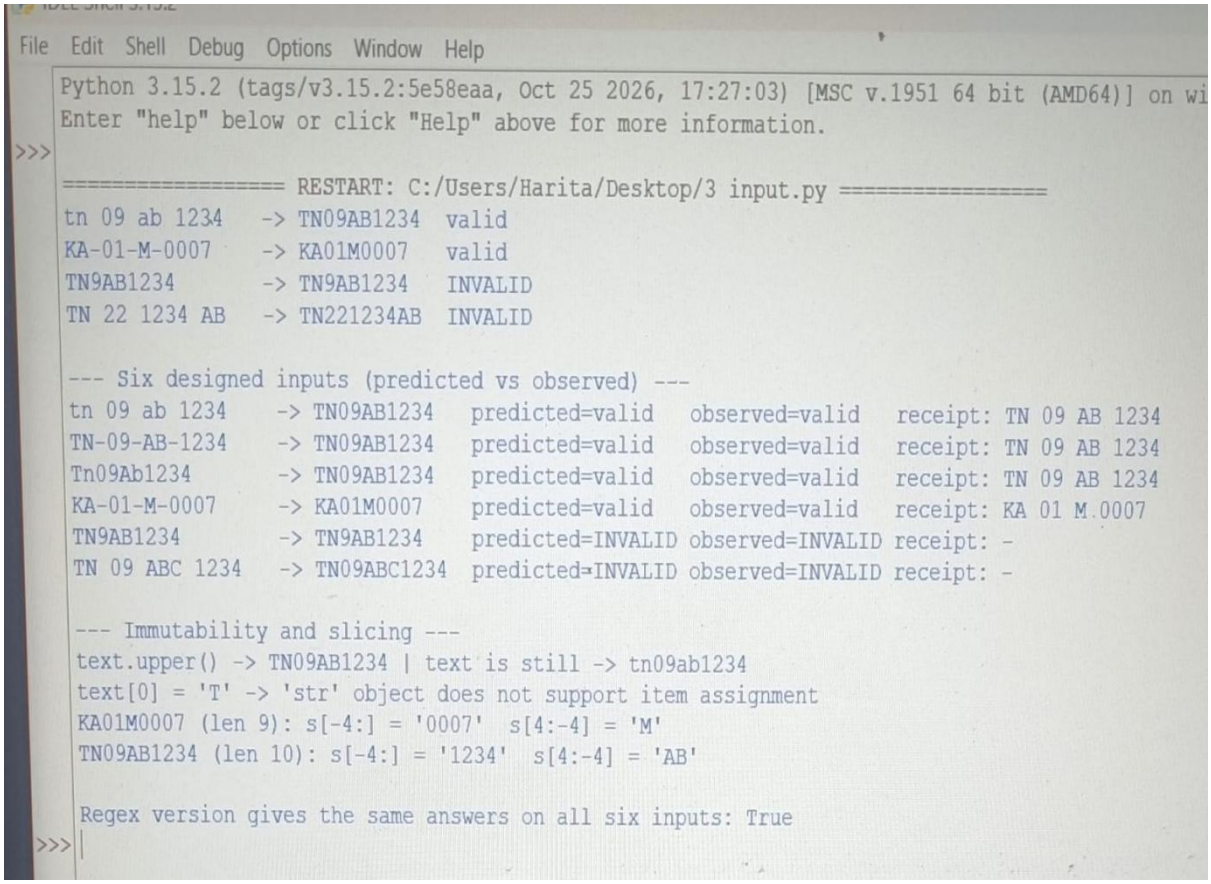
print("\n--- Immutability and slicing ---")
text = "tn09ab1234"
print("text.upper() ->", text.upper(), "| text is still ->", text)
try:
    text[0] = "T"
except TypeError as err:
    print("text[0] = 'T' ->", err)
for s in ("KA01M0007", "TN09AB1234"):
    print(f"{s} (len {len(s)}): s[-4:] = {s[-4:]!r}  s[4:-4] = {s[4:-4]!r}")
```

```python
# ---------- Alternative: regular expression ----------
import re


def is_valid_regex(reg):
    return re.fullmatch(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}", reg) is not None


same = all(is_valid(normalise(r)) == is_valid_regex(normalise(r)) for r, _ in cases)
print("\nRegex version gives the same answers on all six inputs:", same)
```

Output:



Inquiry:

Why strings are immutable?

A Python string cannot be changed after it is created (text[0] = 'T' raises TypeError, shown in the output). Immutability makes strings safe to share, lets them be used as dictionary keys (their hash never changes) and lets Python reuse them.

Why text.upper() does not change text?

Every string method returns a new string and leaves the original unchanged. The change is kept only when the result is assigned, e.g. text = text.upper() (the output shows the original is still lower case).

Slicing-

For a 9-character number such as KA01M0007, s[-4:] returns '0007' and s[4:-4] returns 'M' (a one-letter series). For a 10-character number such as TN09AB1234, s[-4:] returns '1234' and s[4:-4] returns 'AB'. This is why the same slices work for both lengths.

Alternative Concept & Complexity Analysis:

re.fullmatch(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}", reg) replaces the whole is_valid function and gives the same answer on all six inputs (printed in the output). Both versions are O(L) in time for a number of length L, and O(1) extra space apart from the small slices. The regular expression is shorter and the rule is changed by editing one pattern (for example {1,2}); the slicing version is longer but needs no import, can be read step by step, and each check can be debugged separately and given its own error message.

Q4 – Building Long Text: + or join()?

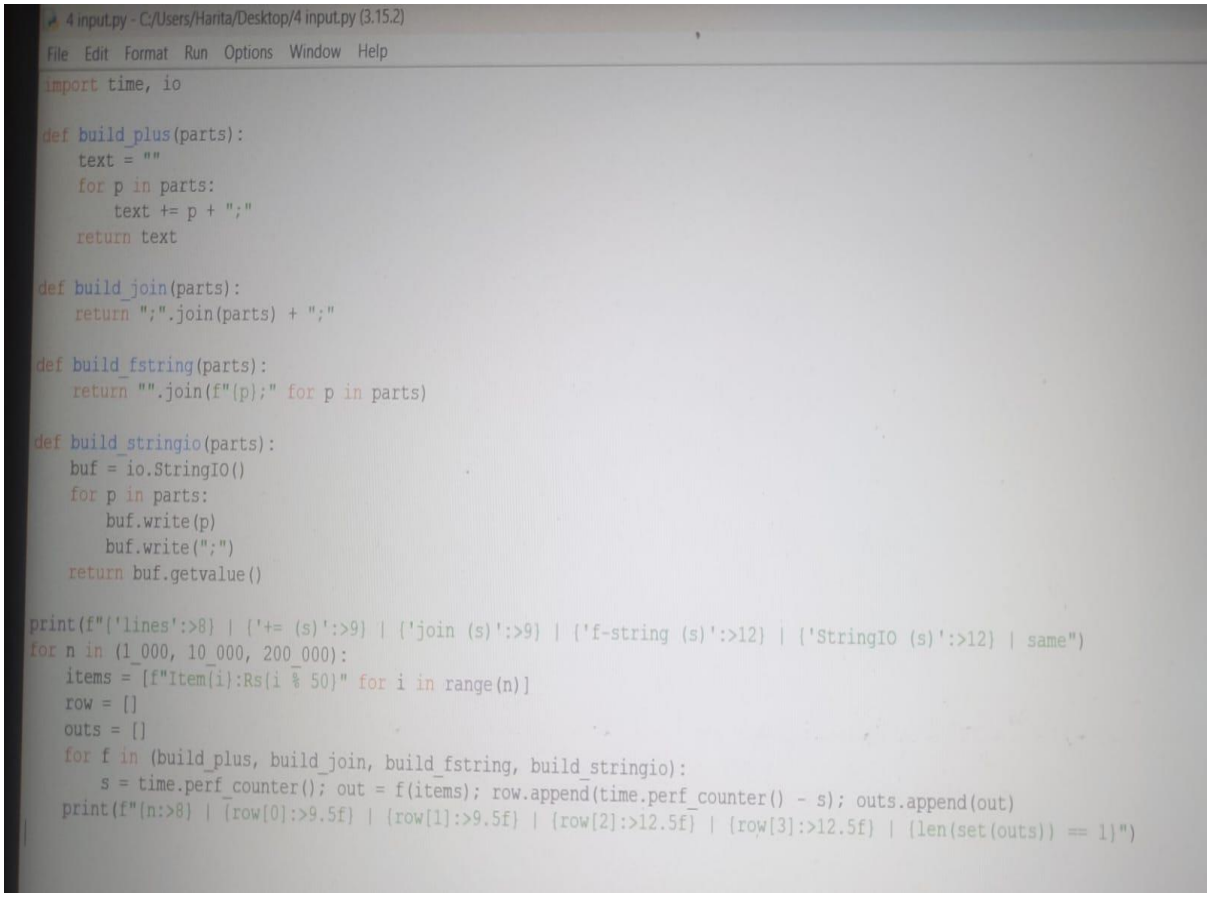
Aim:

To build a long SMS statement using repeated +=, a list followed by join(), an f-string per line (and StringIO), to measure the time for 1,000, 10,000 and 200,000 lines and to explain the result using the immutability of strings.

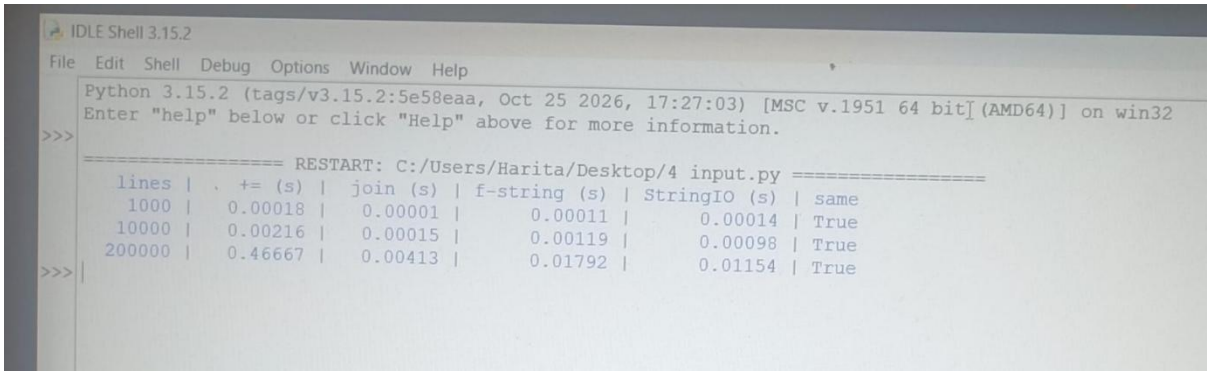
Algorithm:

1. Create the list of statement lines (Item<i>:Rs<i % 50>).
2. Method (a): start with an empty string and add every line with += inside a loop.
3. Method (b): join all the lines with a single ";".join(parts) call.
4. Method (c): build every line with an f-string and join the pieces.
5. Time every method with time.perf_counter() for 1,000, 10,000 and 200,000 lines and check that all texts are equal.
6. Complete TODO 1 (build_join), run the program three times and take the average of each time.

Input:



Output:



Inquiry & Prediction:

I predicted that join() would be faster. A string is immutable, so text += p creates a new string and copies the old text into it every time; for n lines the characters copied grow like 1 + 2 + … + n, i.e. up to $O(n^2)$. join() first calculates the total length, allocates the result once and copies each piece once, which is O(n).

Observation and Explanation:

join() was the fastest at every size, but += was only about two to four times slower, not quadratically slower. CPython has an optimisation: when the string being extended has only one reference, += resizes it in place instead of copying it. This is an implementation detail, not a language guarantee; it disappears when the string is also referenced elsewhere, when the code is written as s = s + p on some objects, or on other Python implementations. join() stays the recommended practice because its O(n) cost is guaranteed and it does not depend on that optimisation. The f-string method with a generator was slower here because it creates one extra small string per line before joining.

Alternative concept & Complexity Analysis:

Using io.StringIO (build_stringio in the program), every line is written to an internal buffer and the text is taken once with getvalue(). Time is O(n), the same as join, and memory is O(n) for the buffer. Writing each line directly to a file keeps only one line in memory, O(1) extra memory, so it is the best choice when the statement does not need to be held as one string. Repeated += is up to $O(n^2)$ characters copied in theory (and only O(n) when CPython's optimisation applies); join() needs the list of parts and the final string in memory, about O(n) extra space.

Q5 – Functions, Parameters and Scope

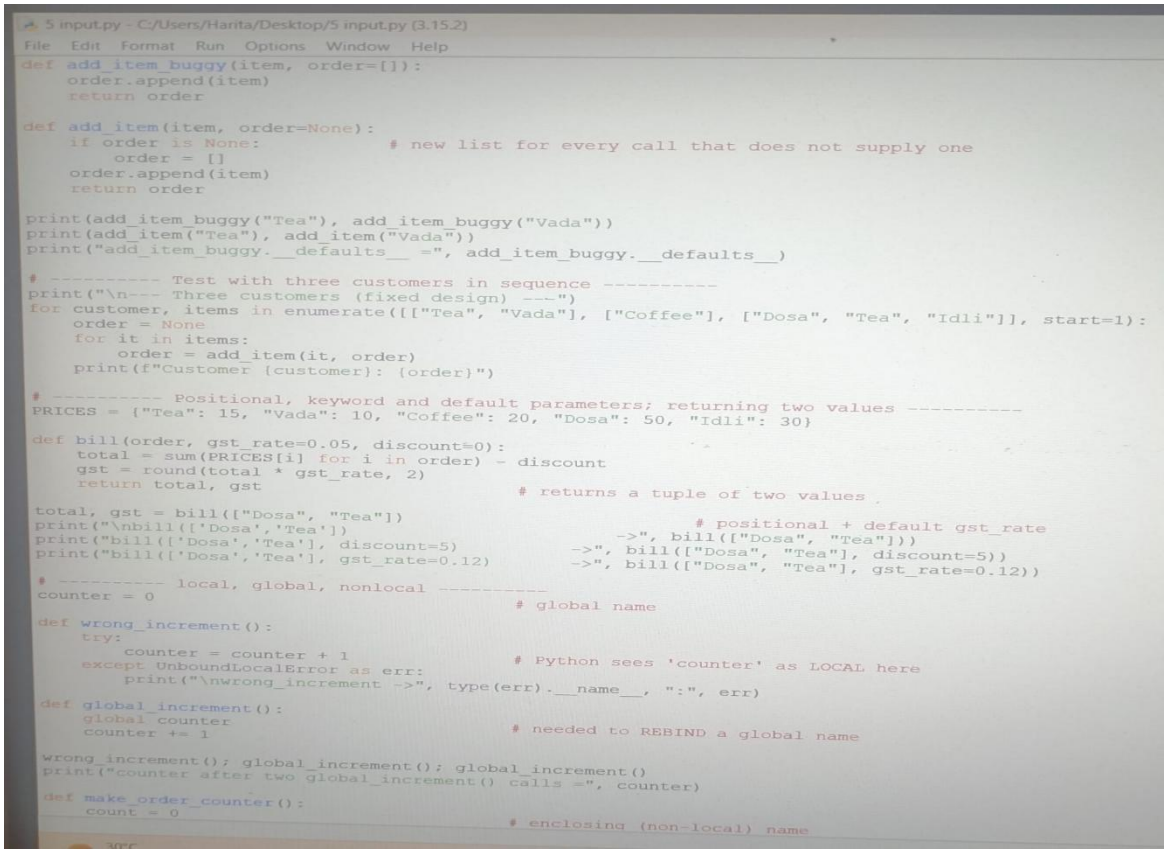
Aim:

To explain and fix the default-argument bug in the canteen order system, to use positional, keyword and default parameters, to return two values from a function and to demonstrate local, global and nonlocal names.

Algorithm:

1. Understand the principle: a default value is created once, when the function is defined.
2. add_item_buggy(item, order=[]) uses one shared list for every call, so every customer's items go into the same list.
3. Fixed design: add_item(item, order=None); if order is None, create a new empty list; append the item and return the list.
4. Test three customers in sequence and show that the orders are independent.
5. Write bill(order, gst_rate, discount) that returns (total, gst) as a tuple.
6. Demonstrate local, global (with the global keyword) and nonlocal names with small examples.

Input:



```
        nonlocal count                          # rebinds the enclosing variable
        count += 1
        return count
    return add_one

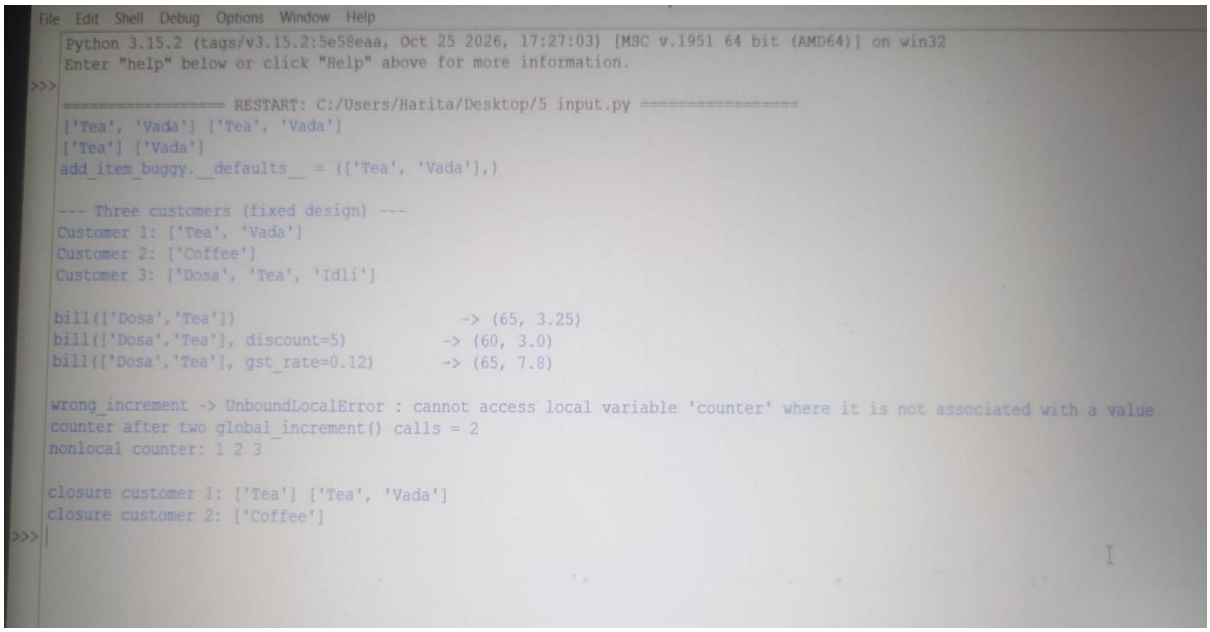
tick = make_order_counter()
print("nonlocal counter:", tick(), tick(), tick())

# ---------- Alternative 1: caller always passes the list ----------
def add_item_explicit(item, order):
    order.append(item)
    return order

# ---------- Alternative 2: closure keeps the order ----------
def new_order():
    order = []
    def add(item):
        order.append(item)
        return list(order)                      # a copy is returned
    return add

c1, c2 = new_order(), new_order()
print("\nclosure customer 1:", c1("Tea"), c1("Vada"))
print("closure customer 2:", c2("Coffee"))
```

Output:



Explanation of the bug:

The default [] is evaluated once, when the def statement runs, and the list is stored inside the function (add_item_buggy.__defaults__). Every call without a second argument uses that same list and appends to it. The output shows __defaults__ = (['Tea', 'Vada'],): the default list itself has grown, so the next customer sees the previous customer's items. The fix uses None as a marker and creates a new list inside the function on every call.

Parameters, return values and scope:

- Parameters allow values to be passed to a function. Python supports positional, keyword and default parameters.
- The return statement sends the result back to the calling function. Multiple values can be returned using a tuple, e.g., return total, gst.
- Scope defines where variables can be accessed:
    1. **Local:** Accessible only inside a function.
    2. **Global:** Accessible throughout the program using the global keyword for modification.
    3. **Nonlocal:** Used to modify variables in an enclosing function using the nonlocal keyword.

Alternative concept and complexity:

(a) The caller always passes the list (add_item_explicit): each call is O(1) per item and the caller owns the order, so there is no hidden state, but the caller must remember to create a new list. (b) A closure (new_order) keeps the list inside an inner function: each add is O(1) per item, one list per customer is kept inside the closure, and a copy is returned for printing, which costs O(m) extra space for m items. Compared with the default None parameter, all three designs have the same O(1) time per item and O(m) space per customer.

Q6 – Recursion and Its Limits

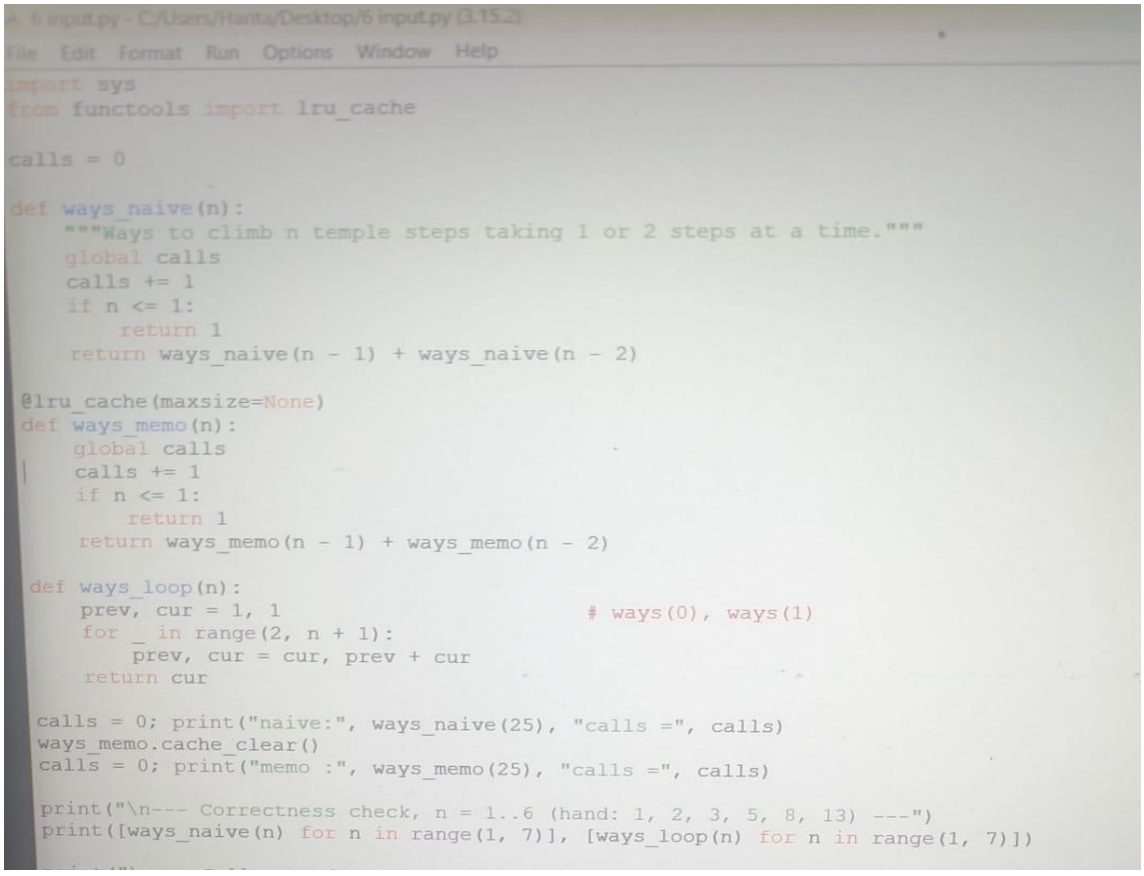
Aim:

To count the number of ways to climb n temple steps (1 or 2 steps at a time) using plain recursion, recursion with memoisation and a loop, to count function calls and to decide how to run it for 1,000 steps.

Algorithm:

1. Base case: if n <= 1, there is exactly 1 way.
2. Recursive case: the last move was 1 step or 2 steps, so ways (n) = ways (n − 1) + ways (n − 2).
3. Memoisation: store every computed ways(n) (functools.lru_cache) and return the stored value when it is asked again.
4. Loop version: keep only the last two values and update them n − 1 times.
5. Count calls for n = 10, 20, 25, 30 and compare with the prediction.
6. Run n = 1,000, observe the RecursionError and decide how to deploy.

Input:



```
6 input.py - C:/Users/Hanta/Desktop/6 input.py (3.15.2)
File  Edit  Format  Run  Options  Window  Help
import sys
from functools import lru_cache

calls = 0

def ways_naive(n):
    """Ways to climb n temple steps taking 1 or 2 steps at a time."""
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)

@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)

def ways_loop(n):
    prev, cur = 1, 1                          # ways(0), ways(1)
    for _ in range(2, n + 1):
        prev, cur = cur, prev + cur
    return cur

calls = 0; print("naive:", ways_naive(25), "calls =", calls)
ways_memo.cache_clear()
calls = 0; print("memo :", ways_memo(25), "calls =", calls)

print("\n--- Correctness check, n = 1..6 (hand: 1, 2, 3, 5, 8, 13) ---")
print([ways_naive(n) for n in range(1, 7)], [ways_loop(n) for n in range(1, 7)])
```

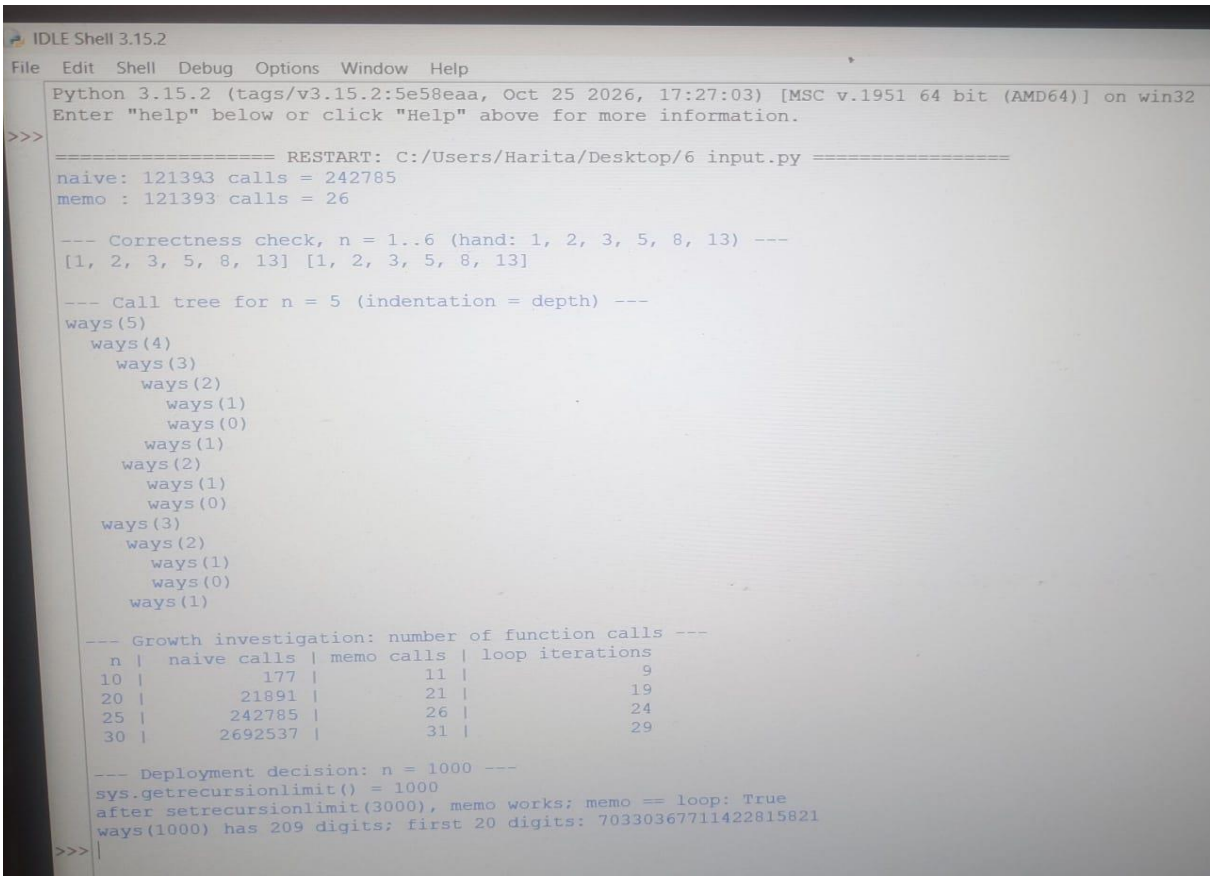
```
print("\n--- Call tree for n = 5 (indentation = depth) ---")
def tree(n, depth=0):
    print("  " * depth + f"ways({n})")
    if n > 1:
        tree(n - 1, depth + 1)
        tree(n - 2, depth + 1)
tree(5)

print("\n--- Growth investigation: number of function calls ---")
print(f"{'n':>3} | {'naive calls':>12} | {'memo calls':>10} | {'loop iterations':>15}")
for n in (10, 20, 25, 30):
    calls = 0; ways_naive(n); c_naive = calls
    ways_memo.cache_clear(); calls = 0; ways_memo(n); c_memo = calls
    print(f"{n:>3} | {c_naive:>12} | {c_memo:>10} | {max(n - 1, 0):>15}")

print("\n--- Deployment decision: n = 1000 ---")
print("sys.getrecursionlimit() =", sys.getrecursionlimit())
ways_memo.cache_clear()
try:
    ways_memo(1000)
except RecursionError as err:
    print("ways_memo(1000) ->", type(err).__name__, ":", err)

old = sys.getrecursionlimit()
sys.setrecursionlimit(3000)
ways_memo.cache_clear()
r_memo = ways_memo(1000)
sys.setrecursionlimit(old)
r_loop = ways_loop(1000)
print("after setrecursionlimit(3000), memo works; memo == loop:", r_memo == r_loop)
print("ways(1000) has", len(str(r_loop)), "digits; first 20 digits:", str(r_loop)[:20])
```

Output:



Correctness check:

N	Ways (hand calculation)
1	1 (1)
2	2 (1+1, 2)
3	3 (ways(2) + ways(1) = 2 + 1)
4	5 (3 + 2)
5	8 (5 + 3)
6	13 (8 + 5)

Why the plain recursion repeats work ?

In the call tree for n = 5 (printed in the output) ways(3) is computed 2 times, ways(2) 3 times and ways(1) 5 times, although each answer never changes. Each call makes two more calls, so the number of calls is $2 \cdot F(n+1) - 1$, which grows by a factor of about 1.618 for each extra step: $O(\varphi^n)$.

Memoisation:

Stores the result of each ways(k) the first time; every later request is a dictionary look-up. Only n + 1 distinct values are computed, so the calls fall from exponential to linear: 242,785 to 26 calls for n = 25 (matches the expected output).

Growth investigation:

My prediction was that the calls multiply by about 1.618 for each extra step, i.e. about 11 times for every 5 steps. The output agrees: 177, 21,891, 242,785 and 2,692,537 calls for n = 10, 20, 25, 30, while the memoised version needs n + 1 calls (11, 21, 26, 31).

Deployment decision (n = 1,000):

sys.getrecursionlimit() is 1000 by default. ways_memo(1000) must keep about 1,000 calls waiting at the same time (and the lru_cache wrapper adds more frames), so Python raises RecursionError. Raising the limit (sys.setrecursionlimit(3000)) makes it work for 1,000 steps, but it only moves the limit: a much larger n can overflow the real C stack and crash the interpreter. **Decision: rewrite with a loop** (O(1) space, no limit, no risk) and keep the memoised version only for explaining the idea. Raising the limit is acceptable only as a temporary measure.

Alternative concept and Complexity Analysis:

Version	Time	Space	n = 1000
Plain recursion	$O(\varphi^n)$	O(n) stack	never finishes; RecursionError
Memoised recursion	O(n)	O(n) cache + O(n) stack	RecursionError at default limit
Loop, last two values	O(n)	O(1)	works: 209-digit answer

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate

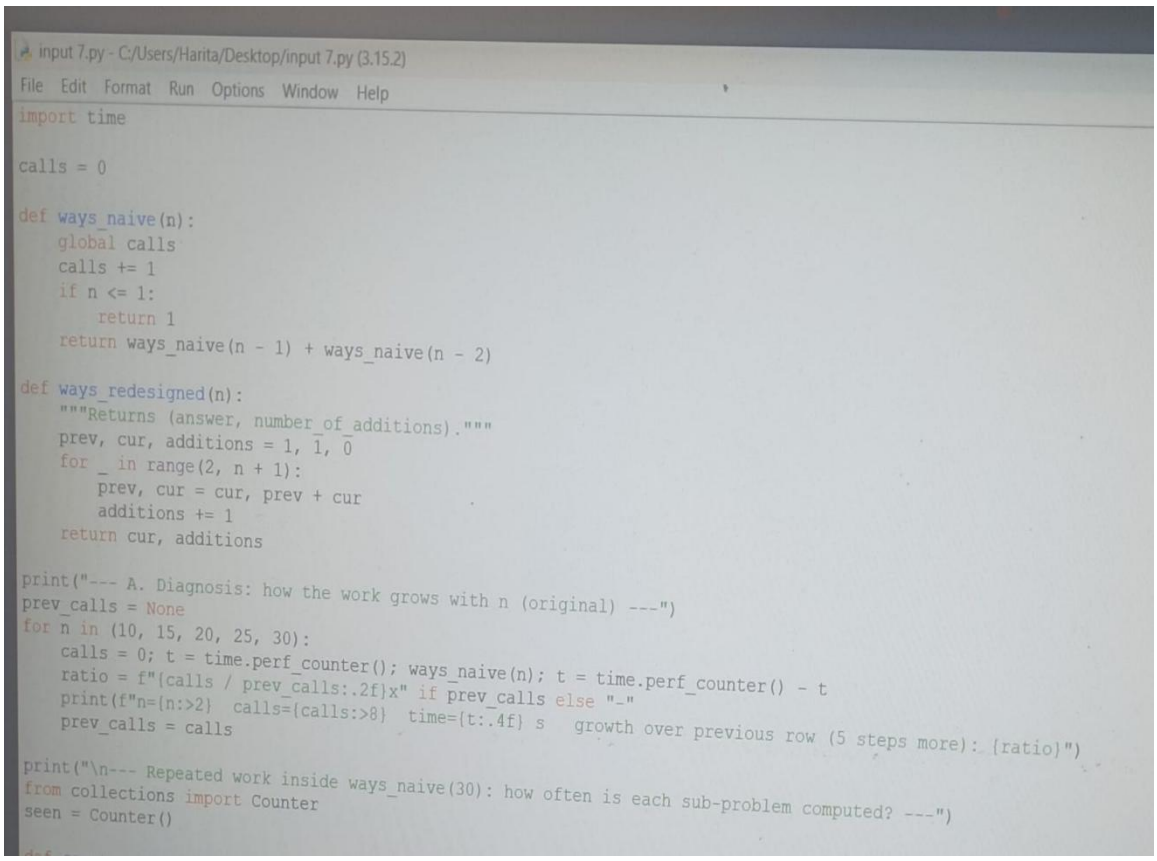
Aim:

To select one program from Q1–Q6 that fails as the input grows (the plain recursive temple-steps program of Q6), diagnose its bottleneck with measured evidence, redesign it without simply calling a built-in function, and validate the improvement.

Algorithm:

1. Select the plain recursive ways_naive(n) of Q6.
2. Measure calls and time for n = 10 to 30 and count how often each sub-problem is computed.
3. State a measurable objective for the redesign.
4. Redesign bottom-up: start with ways(0) = 1 and ways(1) = 1 and compute ways(2), ways(3), … in order, keeping only the last two values.
5. Predict the improvement before implementing.
6. Run the original and the redesigned version on the same inputs, compare calls and times, and evaluate.

Input:



```
def count_subproblems(n):
    seen[n] += 1
    if n > 1:
        count_subproblems(n - 1); count_subproblems(n - 2)

count_subproblems(20)
print("n=20 -> ways(1) computed", seen[1], "times, ways(2):", seen[2], ", ways(5):", seen[5], ", ways(10):", seen[10])
print("distinct sub-problems:", len(seen), "but total calls:", sum(seen.values()))

print("\n--- E. Original vs Redesigned (same inputs) ---")
print(f"{'n':>4} | {'orig calls':>10} | {'new additions':>13} | {'orig time (s)':>13} | {'new time (s)':>12} | same answer")
for n in (10, 20, 25, 30):
    calls = 0; t0 = time.perf_counter(); a = ways_naive(n); t_orig = time.perf_counter() - t0
    t1 = time.perf_counter(); b, adds = ways_redesigned(n); t_new = time.perf_counter() - t1
    print(f"{n:>4} | {calls:>10} | {adds:>13} | {t_orig:>13.6f} | {t_new:>12.8f} | {a == b}")

print("\nRedesign on larger inputs (original cannot finish / crashes):")
for n in (1000, 10000):
    t = time.perf_counter(); r, adds = ways_redesigned(n); t = time.perf_counter() - t
    print(f"n={n:>5}  additions={adds:>5}  time={t:.6f} s  digits in answer={len(str(r))}")

try:
    ways_naive(1000)
except RecursionError as err:
    print("original ways_naive(1000) ->", type(err).__name__)
```

Output:

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.15.2 (tags/v3.15.2:5e58eaa, Oct 25 2026, 17:27:03) [MSC v.1951 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/Harita/Desktop/input 7.py ==========
--- A. Diagnosis: how the work grows with n (original) ---
n=10  calls=     177  time=0.0000 s   growth over previous row (5 steps more): -
n=15  calls=    1973  time=0.0001 s   growth over previous row (5 steps more): 11.15x
n=20  calls=   21891  time=0.0014 s   growth over previous row (5 steps more): 11.10x
n=25  calls=  242785  time=0.0143 s   growth over previous row (5 steps more): 11.09x
n=30  calls= 2692537  time=0.1527 s   growth over previous row (5 steps more): 11.09x

--- Repeated work inside ways_naive(30): how often is each sub-problem computed? ---
n=20 -> ways(1) computed 6765 times, ways(2): 4181 , ways(5): 987 , ways(10): 89
distinct sub-problems: 21 but total calls: 21891

--- E. Original vs Redesigned (same inputs) ---
   n | orig calls | new additions | orig time (s) | new time (s) | same answer
  10 |        177 |             9 |      0.000011 |   0.00000650 | True
  20 |      21891 |            19 |      0.001050 |   0.00000380 | True
  25 |     242785 |            24 |      0.009189 |   0.00000500 | True
  30 |    2692537 |            29 |      0.100802 |   0.00000720 | True

Redesign on larger inputs (original cannot finish / crashes):
n= 1000  additions=  999  time=0.000040 s  digits in answer=209
n=10000  additions= 9999  time=0.001251 s  digits in answer=2090
```

A. Diagnose the bottleneck :

The work is done by the function calls themselves: ways_naive(n) is called $2 \cdot F(n+1) - 1$ times (2,692,537 calls for n = 30). The count that grows with the input is the number of calls, and it multiplies by about 11.09 for every 5 extra steps ($\varphi^5 \approx 11.09$, measured in the first table), so it is exponential. The work is hugely repeated: for n = 20 there are only 21 distinct sub-problems but 21,891 calls; ways(1) alone is computed 6,765 times. The depth of recursion also grows with n, which causes the RecursionError at n = 1,000.

B. Design objective:

For n = 30 reduce the number of operations from 2,692,537 calls to at most n − 1 = 29 additions; give correct answers for every n from 1 to 1,000 (and larger) with no RecursionError; keep extra memory O(1).

C. Revised design:

What changed-

The solution is computed from the smallest sub-problems upwards instead of from the top downwards.

Property exploited-

Overlapping sub-problems (the same ways(k) is needed again and again) and the fact that ways(k) depends only on the previous two values, so older values can be forgotten.

Extra memory-

Only two variables, O(1).

New trade-off-

The code no longer mirrors the mathematical definition as directly; and if many different values of n are asked repeatedly, a stored table (or memoised cache) would reuse earlier answers, while this loop recomputes from the start for every query.

Pseudocode:

```
Algorithm ways_redesigned(n)

 prev <- 1                 // ways(0)
 cur  <- 1                 // ways(1)
 FOR k FROM 2 TO n DO
    next <- prev + cur
    prev <- cur
    cur  <- next
 END FOR
 RETURN cur
```

Flowchart:

Start → Read n → prev = 1, cur = 1, k = 2 → decision "k <= n?" → Yes: next = prev + cur; prev = cur; cur = next; k = k + 1; back to the decision → No: print cur → Stop.

D. Prediction:

The number of operations will fall from about 1.6^n calls to n − 1 additions (for n = 30, from 2,692,537 to 29), and the running time from about 0.15 seconds to a few microseconds. The answers must be identical for every n.

E. Implementation and validation:

The table "Original vs Redesigned" in the output uses the same inputs (n = 10, 20, 25, 30) for both versions; the last column shows that the answers are identical. The measured times are from my computer and will be different on yours, but the counts are the same everywhere.

F. Defend the final design:

Objective achieved:

For n = 30 the operations fell from 2,692,537 calls to 29 additions (about 92,000 times fewer) and the time fell by a factor of tens of thousands; n = 1,000 and n = 10,000 run in microseconds to milliseconds with no RecursionError, and the extra memory is O(1).

New costs:

Less readable as a direct copy of the definition; no reuse of earlier answers across queries; for very large n the cost is in big-number addition (the answer has 209 digits for n = 1,000 and 2,090 digits for n = 10,000).

Where the original may still be preferred:

For very small n or for teaching, where the recursive definition is the clearest explanation and the cost is negligible.